
第 12 章

工程化与持续演进

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

12. 工程化与持续演进

本页是第二部分第 12 章的章首页。前三章解决“会不会用”，本章解决“能不能长期用下去”。

本章概览

AI 工具迭代极快：本章写作基线是 dsh 0.1.5-rc.2，而官方在两周内就发布了多个新版本。真正可迁移的能力不是记住某个命令，而是**建立一套跟着版本走的工程习惯**：锁定版本、定期巡检、记录变更、控制成本、规范协作、按合规提交。

本章把教材自己的维护机制（版本单一事实源、巡检脚本、双周 CI、更新 SOP）作为教学案例讲清楚，并以一个综合项目收尾。

学习目标

- 1. 会为 AI 工具建立“版本单一事实源”并定期巡检（npm dist-tags + GitHub Releases）；
- 2. 会写更新 SOP：发现新版本 → 评估影响面 → 改文档 → 重跑验证 → 提交；
- 3. 会估算与控制 AI 使用成本（模型档位、token 估算、空闲时段、轮次限制）；
- 4. 会制定团队/课程的协作规范（AGENTS.md、评审、会话留档、AI 使用声明）；
- 5. 完成综合项目 Project 3：用 Agent 完成一次端到端数据分析并提交报告。

本章结构

小节	内容	状态
01 版本与时效管理	版本单一事实源、巡检、双周 CI、更新 SOP	已交付
02 成本、协作与合规	成本治理、团队规范、数据分级与合规	已交付
03 综合项目 Project 3	端到端数据分析项目（含 AI 使用声明与评分标准）	已交付

资源导航

资源	位置	说明
参考资料	references.md	版本管理、CI、成本与合规资料
教学说明	teaching.md	教师用：项目布置、评分与常见问题
维护件	docs/build/dsh_version.yaml、docs/build/check_dsh_release.py、docs/build/dsh_更新 SOP.md	教材自身的版本巡检机制
期末项目	../教学资源/期末大作业/	Project 1 / 2 / 3 说明与模板

01 版本与时效管理

本节的定位：AI 工具迭代极快，教材写作基线是 dsh 0.1.5-rc.2，而官方在两周内就发布了多个版本。
真正可迁移的能力，是**建立一套跟着版本走的工程习惯**。

本节目标

- 1. 会为 AI 工具建立“版本单一事实源”；
- 2. 会做版本巡检（npm dist-tags + GitHub Releases + 官方文档）；
- 3. 会写更新 SOP：发现新版本后按固定流程改文档、重验证、再提交；
- 4. 知道“时效标记”的作用：让过期内容可以被一次性找出来。

1.1 为什么需要版本管理

现象	后果
命令/参数随版本变化	学生照教材敲命令报错
界面与菜单调整	截图与步骤说明失效
模型名称与价格调整	成本估算失真

结论：**凡是“会变”的内容，都要有单一事实源 + 定期巡检 + 更新流程**，而不是靠记忆。

1.2 版本单一事实源

教材把版本信息集中在一个文件里：docs/build/dsh_version.yaml

字段	作用
baseline.version / released_at / checked_at	基线版本与核对日期
baseline.node	Node.js 版本要求
upstream.*	官方仓库、文档站、Releases、定价页、开放平台链接
watch.dist_tags	需要盯的发布渠道（latest / next / alpha）
review_on_change	一旦版本变化，必须复核的文件清单

要点：把“需要改哪些文件”写进事实源，更新时就不靠回忆。

1.3 版本巡检：三条命令 + 一个页面

手段	命令 / 入口	看什么
npm 渠道	npm view @deepseek-ai/dsh dist-tags	latest / next / alpha 各是什么版本
GitHub 发布	仓库 Releases 页	每个版本的中文更新说明（含破坏性变更）
本机版本	dsh --version	本机装的是哪个版本
官方文档	文档站	命令、界面、配置的最新说明

1.4 双周巡检与自动提醒

教材仓库用 GitHub Actions 做**双周巡检**（每月 1 日与 15 日）：[.github/workflows/dsh-release-watch.yml](#)

- 发现新版本 → 自动开 Issue（含更新 checklist）；
- 已存在同名开启中的 Issue → 不重复创建；
- 巡检报告作为 artifact 存档。

你也可以给自己用：把巡检命令写进日历提醒，或写一个本地脚本定期跑。

1.5 更新 SOP（八步）

1. 跑巡检脚本 `check_dsh_release.py`（或手工执行 1.3 的三条命令）；
2. 读 Release notes，标记**可能影响教材**的条目（命令、界面、模型、价格）；
3. 用“时效内容（基线”作为关键词，一次找出所有需要复核的段落；
4. 按 `review_on_change` 清单逐文件修改；
5. 更新 `dsh_version.yaml`（版本、核对日期、复核记录）；
6. 复核/重截受影响的截图（第 9 章 03 节那 4 张）；
7. 重跑构建与校验（`validate_book` / `validate_code` / `check_indexes` / `check_pdf_layout`）；
8. 提交并关闭巡检 Issue。

1.6 时效标记约定

正文里所有易变段落都以固定格式开头，例如：

时效内容（基线 v0.1.5-rc.2 / 2026-09-19）：本节涉及版本号、命令、界面与价格。

好处：**升级时只需一次 grep**，不必通读全书。

本节小结

1. 版本管理的三步：单一事实源、定期巡检、固定 SOP；
2. 把“要复核哪些文件”写进事实源，而不是记在脑子里；
3. 时效标记让过期内容可被一次性定位；
4. 这套机制不仅适用于教材，也适用于你自己的科研代码与工具链。

思考题

1. 为什么“破坏性变更”值得单独标记？请举一个会破坏教材示例的例子；
2. 如果你的课题依赖某个快速迭代的库，你会把本节哪三条做法搬过去？
3. 巡检发现新版本后，什么情况下应当**暂不升级**？

练习与延伸阅读

- 练习：[exercises/assignment.md](#) 第 10 题（写一份属于你的更新 SOP）；
- 参考：[docs/build/dsh_version.yaml](#)、[docs/build/check_dsh_release.py](#)、[docs/build/dsh_更新 SOP.md](#)。

02 成本、协作与合规

本节的定位：把“个人会用”升级为“团队/课程可管理”：成本怎么控、协作怎么分工、数据怎么合规。

本节目标

1. 会为课程/团队设定 AI 使用预算与限额；
2. 会制定协作规范（AGENTS.md、评审、留档、提交）；
3. 会对数据分级并做脱敏决策；
4. 知道第三方插件与自动化脚本的风险边界。

2.1 成本治理

措施	做法	说明
预算与额度	每个 Key 设低额度；练习与个人长期使用分开	平台可按 Key 查看用量
模型档位	默认 deepseek-flash；仅复杂推理用 pro	价格差 3–4 倍
时段调度	批量任务放空闲时段	空闲为高峰半价
轮次控制	一次把判据说清；长任务定期收口	返工是最贵的开销
用量记账	每次任务记录耗时与花费（见第 11 章 06 节）	便于复盘与申报

2.2 协作规范（写进仓库）

1. AGENTS.md 进版本库：项目约定、安全约定、验收要求；
2. 任务卡进 Issue/文档：目标、判据、证据要求、提交形态；
3. 评审独立：实现与评审分离（人或评审代理），评审意见逐条回应；
4. 会话留档：关键任务导出会话（/export），与代码一起归档；
5. 小步提交：一次提交只做一件事，提交信息写清“AI 参与程度”。

2.3 数据分级与合规

级别	例子	处置
可公开	课程示例数据、公开数据集	可直接交给云端模型
内部	课题组内部整理的中间数据	脱敏后可用；必要时用本地模型
敏感	个人信息、医疗、涉密、未发表成果	不上传 ；本地处理或改为合成数据

脱敏三步：去标识（姓名/学号/身份证）→ 粗粒度化（精确日期改月份）→ 抽样检查（确认无法反推个人）。

2.4 第三方插件与自动化风险

- 插件来自社区：安装前看维护活跃度、权限要求与代码来源；
- 自动化脚本（定时任务、webhook）：确认它会把什么数据发到哪里；
- 依赖升级：先在小范围验证，再全量使用（与第 01 节版本管理呼应）。

2.5 与学术诚信的衔接

第 10 章 04 节给了 AI 使用声明模板；团队/课程层面再补三条：

1. 提交物必须包含“可复现证据”（脚本 + 输出 + 会话记录）；
2. 结论部分必须由人撰写并署名；
3. 违规处理公开透明（例如：未声明使用 AI 视为学术不端）。

本节小结

1. 成本治理的五条：额度、档位、时段、轮次、记账；
2. 协作规范的五条：AGENTS.md、任务卡、独立评审、留档、小步提交；
3. 数据分三级，敏感数据不上传；脱敏三步走；
4. 插件与自动化要评估来源与数据流向。

思考题

1. 为什么“给每个 Key 设额度”比“月底看账单”更有效？
2. 团队里如何让“评审独立”真正落地，而不是走形式？
3. 你手上哪份数据属于“内部”级别？写出它的脱敏方案。

练习与延伸阅读

- 练习：exercises/assignment.md 第 11 题（协作规范草案）；
- 参考：第 10 章 04 节（AI 使用声明与学术诚信）。

03 综合项目 Project 3

本节的定位：把第二部分的方法用在一个完整项目上。Project 3 与期末大作业 Project 1/2 并列，可直接沿用它俩的数据与报告规范。

项目目标

用 Agent 完成一次端到端数据分析，并提交可复现的产物与 AI 使用声明。

选题（任选其一）

题目	数据	任务
A 电商用户分析	期末项目 1 的数据	清洗 → 用户分群（聚类）→ 关键指标与图表 → 结论
B 交通网络分析	期末项目 2 的数据	建图 → 中心性/最短路/社区 → 可视化 → 结论
C 自选（需教师同意）	你自己的数据（须合规）	自定分析目标与判据

交付物清单

- 1. 项目目录：AGENTS.md、src/（脚本）、output/（结果与图）、report/（LaTeX 源码与 PDF）；
- 2. 报告 PDF：含数据、方法、结果、局限、AI 使用声明；
- 3. 会话记录：关键任务的 /export 存档；
- 4. 实验记录表（第 11 章 06 节格式）：耗时、花费、人工修改。

阶段安排（建议 2 周）

阶段	产出	对应方法
1 选题与计划	任务卡 + 计划 (plan mode)	第 10 章 01–02 节
2 数据与清洗	clean_rules.yaml + verify_clean.py	第 11 章 02 节
3 分析与建模	分析脚本 + 判据断言	第 11 章 03–04 节
4 评审与修复	review.md（意见逐条回应）	第 11 章 04 节与 05 节
5 报告与提交	LaTeX PDF + AI 使用声明	第 10 章 04 节、第 0 章 LaTeX

评分标准（建议）

维度	权重	看什么
分析正确性	30%	判据是否成立（行数守恒、交叉验证、检验前提）
验证充分性	25%	断言/复核是否覆盖关键风险（泄漏、异常值、稳健性）
使用规范性	20%	任务卡、AGENTS.md、权限与数据红线、评审闭环
报告质量	15%	结构、图表规范、局限与不确定性
声明与记录	10%	AI 使用声明完整、会话记录与实验记录可追溯

常见扣分点

- 只报“最好结果”，不报基线、区间与失败尝试；
- 报告数字与 output/ 对不上；
- 没有逐条回应评审意见；
- 用 Agent 直接生成结论段落，且无法解释来源。

本节小结

1. Project 3 复用期末项目的数据与报告规范，只是多了“AI 使用规范”这一层；
2. 五个阶段各有方法出处，照着做即可；
3. 评分看的是“可信度”，不是“用了多少 AI”。

思考题

1. 为什么评分把“验证充分性”放在与“正确性”接近的权重？
2. 如果你的分析结论与 Agent 的建议相反，你会怎么处理与记录？
3. 写出你打算做的 Project 3 的一句话目标与两条验收判据。